

Overall operation. I plan to build a Python application that dispatches ambulances to emergency calls using the simulation data. For each call, it will compare the fastest route from every ambulance's staging location, including traffic delays, and select the ambulance with the lowest travel time. It will append the call ID, call type, call location, selected ambulance, route, and travel time to `/var/log/ambulance_call_log.csv` as comma-separated name-value pairs. After logging the dispatch, it will reset the ambulance to its staging location and process the next call. Both prototypes will follow this design, with different routing algorithms that I will choose later. Embedded counters will measure the total execution time spent finding routes.

Simulation files. I will use Python's `csv` module to read the four files. The `ambulance.csv` file provides ambulance numbers and staging locations, `call_priority.csv` connects call types to priorities, and `calls.csv` provides call IDs, locations, and types. The `location_network.csv` file provides connections between locations, distances, travel times, and traffic delays. I will store the network as a directed graph because travel values can differ by direction. Each connection's cost will be its travel time plus traffic delay. Before dispatching, the application will check required fields, convert numeric values, and verify that call types and locations match the supporting data.

Call priorities. I will process priority 1 calls first, followed by priority 2 and then priority 3. The application will look up each call's priority using its call type and keep calls with the same priority in the order received. Since `calls.csv` has no arrival timestamps, I will use its row order as the arrival order. For example, a later priority 1 call will be handled before an earlier priority 2 call, but it will stay behind earlier priority 1 calls. Both prototypes will use the same call order.